

Aperture — Development Roadmap

Version: v1.0

Purpose: Master Engineering Roadmap

1. Introduction

This document defines the implementation roadmap for Aperture.

Unlike the Product Requirements Document, which defines **what** Aperture should become, this document defines **how the platform evolves** from an empty repository into a production-ready application.

Each phase is intentionally designed to:

- produce a deployable release
- teach meaningful engineering concepts
- progressively increase architectural complexity
- avoid premature optimization
- maintain a working application at every stage

Every phase concludes with a Git tag and release milestone.

Engineering Philosophy

Development follows one core principle:

Build the simplest system that can support today's requirements while leaving room for tomorrow's growth.

This means:

- Modular Monolith first
 - Infrastructure second
 - AI later
 - Scale only when necessary
-

Development Lifecycle

Every phase follows the same lifecycle.

Planning

↓

Implementation

↓

Testing

↓

Documentation

↓

Deployment

↓

Release

↓

Retrospective

No phase is considered complete until:

- Features are implemented
- Tests pass
- Documentation is updated
- CI passes
- Application is deployed

Phase 0 — Engineering Foundation

Version

v0.1.0

Goal

Establish a production-grade engineering environment before writing business logic.

This phase has no user-facing functionality.

Instead, it focuses entirely on engineering quality.

Objectives

Create a codebase that:

- is reproducible
 - is testable
 - is deployable
 - is maintainable
-

Features

Repository

- Initialize Git repository
 - Monorepo structure
 - GitHub repository
 - README
 - LICENSE
 - CONTRIBUTING
-

Developer Environment

- Python environment
 - Node environment
 - Docker
 - Docker Compose
 - Environment variables
 - Local development scripts
-

Backend

- FastAPI skeleton

- Health endpoint
 - API versioning
 - Dependency injection setup
 - Configuration management
-

Frontend

- Next.js
 - Tailwind
 - TypeScript
 - ESLint
 - Prettier
-

Quality

- Ruff
 - Black
 - mypy
 - ESLint
 - Prettier
-

GitHub

- Branch protection
 - Pull request templates
 - Issue templates
-

CI/CD

- GitHub Actions
 - Lint
 - Test
 - Build
 - Docker image generation
-

Documentation

Create:

- PRD
- Roadmap

- HLD
 - LLD
-

Deliverable

A fully reproducible engineering foundation.

Technical Concepts

- Docker
 - CI/CD
 - GitHub Actions
 - Monorepo
 - Dev Containers
 - Environment Management
-

Interview Topics

- Why Docker?
 - Why CI?
 - Why Monorepo?
 - Why FastAPI?
-

Phase 1 — User Platform

Version

v0.2.0

Goal

Allow users to securely create accounts and maintain personal profiles.

Features

Authentication

- Registration
 - Login
 - Logout
 - JWT
 - Refresh Tokens
 - Password Hashing
-

OAuth

- Google Sign-In
-

Profiles

- Avatar
 - Bio
 - Username
 - Preferences
-

Security

- Route protection
 - Role system
 - Input validation
-

Frontend

- Login page
 - Signup page
 - Settings page
-

Deliverable

Users can securely create and manage accounts.

Concepts Learned

- Authentication
 - Authorization
 - JWT
 - OAuth
-

Interview Topics

- JWT vs Sessions
 - Password hashing
 - OAuth flow
-

Phase 2 — Metadata Platform

Version

v0.3.0

Goal

Turn Aperture into a usable movie and TV database.

Features

TMDb Integration

- API client
 - Rate limiting
 - Metadata ingestion
 - Synchronization
-

Content

- Movies
 - TV Shows
 - Seasons
 - Episodes
-

Pages

- Detail pages
 - Cast
 - Crew
 - Images
 - Trailers
-

Search

- Title search
 - Person search
 - Filters
-

Caching

- Metadata cache
 - Image optimization
-

Deliverable

Users can browse movies and TV shows.

Concepts

- REST APIs
 - Third-party integrations
 - Caching
 - Data normalization
-

Interview Topics

- Why TMDb?
 - Why normalize external data?
 - Cache strategy
-

Phase 3 — Personal Library

Version

v0.4.0

Goal

Allow users to organize their viewing history.

Features

- Watchlist
 - Watched
 - Favorites
 - Custom Lists
 - Diary
 - Rewatch tracking
-

Deliverable

Users have a persistent personal library.

Interview Topics

- Database relationships
 - State synchronization
 - List modeling
-

Phase 4 — Reviews & Community

Version

v0.5.0

Goal

Introduce social interaction.

Features

- Ratings
 - Reviews
 - Spoiler support
 - Likes
 - Comments
 - User following
 - Activity feeds
-

Deliverable

The platform becomes a social application.

Interview Topics

- Feed generation
 - Pagination
 - Moderation
-

Phase 5 — Streaming Integration

Version

v0.6.0

Goal

Help users discover where content is available.

Features

- Watch providers
- Region filtering
- Streaming badges
- Rent/buy support

Deliverable

Aperture becomes useful before users even choose a movie.

Interview Topics

- Data synchronization
 - External API reliability
 - Background refresh jobs
-

Phase 6 — Search Platform

Version

v0.7.0

Goal

Replace basic search with a production-quality search experience.

Features

- Full-text search
 - Autocomplete
 - Advanced filtering
 - OpenSearch integration
 - Search analytics
-

Deliverable

Fast, scalable search.

Interview Topics

- Search indexing
- Why OpenSearch?
- Full-text search vs SQL

Phase 7 — Recommendation Engine

Version

v0.8.0

Goal

Introduce intelligent movie discovery.

Features

- Similar movies
 - Metadata similarity
 - Editorial recommendations
 - Trending algorithm
 - Personalized homepage
-

Deliverable

Recommendations become a core experience.

Interview Topics

- Recommendation strategies
 - Cold start problem
 - Ranking systems
-

Phase 8 — Semantic Discovery

Version

v0.9.0

Goal

Move beyond genre-based discovery.

Features

- Semantic enrichment
 - Mood tagging
 - Theme extraction
 - Natural language search
 - Emotional Discovery Engine
-

Deliverable

Users can search using intent instead of metadata.

Interview Topics

- Embeddings
 - Vector search
 - Semantic similarity
-

Phase 9 — AI Platform

Version

v1.0.0

Goal

Introduce Aperture's signature AI features.

Features

- Cinematic DNA
 - AI Taste Matching
 - User embeddings
 - Movie embeddings
 - AI-generated collections
-

Deliverable

Aperture now offers experiences unavailable on traditional movie platforms.

Interview Topics

- Embedding pipelines
 - Personalization
 - Recommendation evolution
-

Phase 10 — Movie Spaces

Version

v1.1.0

Goal

Create niche communities around cinema.

Features

- Spaces
 - Discussions
 - Collaborative lists
 - Community moderation
 - Space discovery
-

Deliverable

Community expands beyond individual reviews.

Interview Topics

- Community architecture
 - Moderation
 - Feed ranking
-

Phase 11 — Production Hardening

Version

v1.2.0

Goal

Prepare the platform for real users.

Features

- Performance optimization
 - Redis caching
 - Background jobs
 - Monitoring
 - Logging
 - Metrics
 - Rate limiting
 - Security improvements
 - API optimization
-

Deliverable

Production-ready MVP.

Interview Topics

- Observability
 - Scaling
 - Performance tuning
 - Security
-

Phase 12 — Cloud Evolution

Version

v2.0.0

Goal

Prepare the platform for large-scale production.

Features

- Infrastructure as Code
 - Terraform
 - AWS migration
 - CDN
 - Load balancing
 - Read replicas
 - Object storage
 - Horizontal scaling
-

Deliverable

Cloud-native architecture capable of supporting significant growth.

Interview Topics

- Scalability
 - Cloud architecture
 - Disaster recovery
 - Cost optimization
-

Continuous Work (Across Every Phase)

These are never considered "finished" and should evolve throughout development:

- Documentation
- Testing
- Performance optimization
- Security reviews
- Accessibility improvements
- UI/UX polish
- Refactoring
- CI/CD enhancements
- Monitoring
- Technical debt reduction

Definition of Done

A phase is complete only when:

- Features are implemented.
- Tests pass.
- Documentation is updated.
- CI pipeline succeeds.
- Application is deployed.
- Release notes are written.
- Git tag is created.
- Technical debt is reviewed.

Roadmap Evolution

This roadmap is intended to evolve.

New features should only be introduced if they align with the Product Requirements Document and do not compromise the platform's core philosophy.

The roadmap should remain focused on delivering meaningful, production-quality software rather than accumulating features for the sake of complexity.